

Voronoi Partitioning for Decentralized Coordination - To manage multi-UAV operations, Voronoi partitioning divides the environment into non-overlapping regions, assigning each drone a specific area to minimize redundancy. This technique enhances swarm efficiency while maintaining robust coverage and safety.

Robust Connectivity - Ensuring uninterrupted communication between drones and the ground station is vital. Implementing strategies like multi-Q learning helps drones prioritize paths that maintain connectivity while adapting to terrain-induced challenges, such as mountainous regions or areas with limited satellite coverage.

Object Detection and Data Collection – Ideally, this stage would be efficiently covered by the other VITALs group, but the data passed on from the image processing stage can be used. High-quality data collection ensures the swarm gathers actionable insights to support decision-making and mission goals.

James

Swarm Movement Introduction

For our project, we are tasked with utilizing and enabling an LLM to successfully command a Drone Swarm to maneuver synchronously in real-time based off of Computer Vision results and text input from a user. This is a complex system of Moving parts, so to speak, that requires the use of existing Drone Movement knowledge to the best of our ability.

For both simplicity, and more importantly, system efficiency and success at locating victims, we should default Drone Swarm Movement to optimal search algorithms and Movement patterns as understood by the Drone Swarm community. The LLM we create will work with existing algorithms in order to maximize search efficiency, utilizing the natural

advantages of the drone's high ground, the breadth of camera angles and area coverage a drone swarm provides, and automatic Computer Vision-based image/video scanning in order to make Search-and-Rescue efforts quicker and safer, potentially saving lives.

As a core tenet of the MVP agreed upon by Joe Rivera from Lockheed Martin, our sponsor, we must have the LLM direct this Movement. Therefore, we must train the LLM to identify under which conditions different search algorithms should be used, and correctly implement a function for the LLM to command the drones to follow that Movement by inputting a series of coordinates in the MissionPlanner software interface.

This identifies 4 main steps in implementing Swarm Movement in Search and Rescue: First, Identifying optimal pre-determined Search algorithms and patterns. Second, Training the LLM using Reinforcement Learning to respond with the correct search algorithm based upon different user prompts. This entails rewarding the system when it responds with a correct search path, and punishing it when it doesn't, to skew the results towards the most efficient search patterns. Third, learning how to allow objects flagged by Computer Vision to be input into the Swarm Movement LLM and place objective coordinates in MissionPlanner accordingly. Fourth, researching how to most rapidly send coordinates based upon the search pattern from the Ground Station LLM with minimal lag to MissionPlanner and then the Drone Swarm.

The added complexity for the Search and Rescue use case comes from having to communicate with several real endpoint devices transmitting real-time data, transmitting and receiving real-time locator signals like GPS, and processing initial computer vision-scanned images on the drone to flag to the more powerful Ground Station. The time element is not insignificant either, as human lives are often endangered when Search-and-Rescue

operations are being conducted. An efficient and timely search pattern, effective drone hardware like cameras and processors, as well as a precise Ground Station that can confirm flagged objects and accurately use new data to help the search.

The movement of swarms in existing research

The Movement of Swarms in technical fields can be referred to as Swarm Intelligence, the collective operation of many simpler agents to work together and achieve a larger objective, such as Group or multiple-agent specialized Movement, engaging an objective, or releasing many payloads.

In this case, our application of Swarm Intelligence as directed by an LLM, requires a comprehensive array of modern Swarm search-and-detect algorithms. Swarm Intelligence has a basis in simple Artificial Intelligence like Decision Trees in the world of Video Games with early games such as Starcraft or World of Warcraft using many simple computer agents to achieve tasks within game simulations. Now the modern and cutting-edge Swarm AI architectures have direct applications in Search-and-Rescue and are the basis for real-world technology-based swarms.

As described in the article “How can you design and implement drone swarm algorithms?”, there are two kinds of swarm algorithms, consensus-based and bio-inspired. The consensus-based type is based upon exchanging information with a set of drones between the “peers”, influenced by visual range and how much information the swarm can gather together. This type of algorithm fits our exact need as a project, and we do not need a swarm inspired by how fish move when we have drones unobstructed in the air.

Similar to origins of Swarm Intelligence in video games, and as described by Consensus-based algorithms, we must make sure each Drone is aware of its location and the location of other drones, avoid collision, and know the location and direction of both objectives and next coordinates in

correlation with other drone movement within the swarm. They must also be adaptable to the environment and obstacles in a fluid situation, dealing with wind, water, and visual impairments to the Computer Vision-aiding camera gimbal.

Swarm Movement Algorithms

Now we need to determine the best search algorithms. Our MVP use case assumes a “normal” Search-and-Rescue scenario, searching for a person in the wilderness, with other areas and types of Search=and=Rescue as stretch goals. However according to the research paper out of West Point, “An Investigation of Search Algorithms for Aerial Reconnaissance of an Area Target”, the size of target greatly affects which search pattern should be used by drones. The researchers modified the existing search patterns to optimize for multiple target sizes. Taken from that research is the below figure, which shows Normal and Modified search patterns for the Horizontal Sweep (also called a Lawnmower pattern), Expanding Spiral, and Lissajous Search.

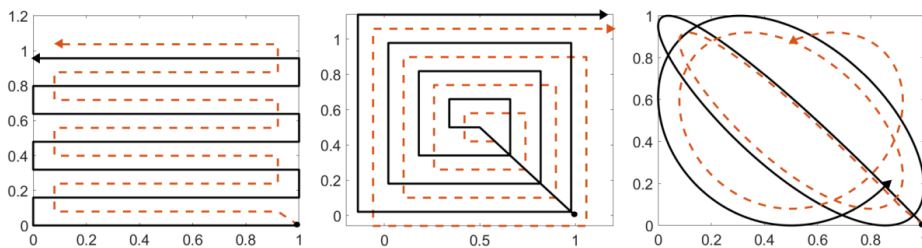


Figure 7: Standard and Modified Deterministic Search Paths

The research determined that for a large search area with a small target, such as a human, a Lissajous Search was the most effective for an active Search-and-Rescue operation utilizing aerial drones. Given an unlimited time, the Horizontal Sweep would cover the entire area in the shortest

amount of time, but Lissajous is able to cover more ground early in the search.

However, these are deterministic search paths, which do not account for changing a search path mid-flight. Therefore, we must utilize calculations based upon smaller search areas centered on points of interest in addition to the larger search pattern. Since we are working with a swarm, leaving 1 or 2 drones to perform the smaller search while having the rest continue the larger search is our current plan, however that is open to change should research indicate otherwise.

The Lissajous deterministic path acts as a good default for a large wilderness search, likely using the drones to expand the width of the line in an optimal drone formation to maximize visibility and minimize overlap between drone camera coverage. However, for a condensed search area, which occurs after finding an input such as a drone finding an abandoned vehicle, it would be more effective at that time to switch to using the Expanding Spiral outward from the point of contact, in this case the vehicle. At any time a human operator can request the LLM to change a decision or use a different known search pattern, in the case that the operator believes the LLM has made a faulty decision, allowing for redundancy in the case of system malfunction or unanticipated edge cases.

Should the drones cover the smaller area without finding anything, the LLM should have those drones rejoin and continue the larger search path, with the larger search path now skirting the heavily searched spiral area. Should more traces be found during the small search, or paths that match a heat map, those should always be followed before defaulting back to the larger search. In the case of a successful identification, the drones continuing the larger search would either rejoin the other drones, or return to the Ground Station.

Jan Hendrick's Search & Rescue landscape research

We have also found research conducted earlier this year on Search Paths of autonomous drones being used in the Scottish Highlands by their police forces, a paper called “Deep Reinforcement Learning for Time-Critical Wilderness Search and Rescue Using Drones”, led by Jan-Hendrick Ewers of the University of Glasgow. We’ve reached out and gotten in contact with Jan-Hendrik, sharing the MVP and prospective plan for our project, and inquiring about the details of their related research.

Their research combined Deep Reinforcement Learning and Probability Distribution Maps, similar to heat maps used in some current drone search methods, and developed specific training algorithms for the drones. They calculate the seen area from the drone by a fixed downward camera, similar to how our drones will be configured. They used an existing reward algorithm that balances Reward, where the system is learning to take the quickest path to a previous found target, and Entropy, where the system will explore new pathways based off probability equations over the Probability Distribution Map.

The team did fix multiple variables like drone height, however the research team successfully demonstrated greater success over the Horizontal Sweep/Lawnmower method by multiple metrics, like Distance To Find, Percentage of Targets Found, and Probability of Finding the Target over Distance. However the training for this was also set over a flat landscape with a height constraint, so this needs to be modified to fit a better real-world use case as opposed to a flat forest or plain before we can use it.

We hope to use our connection with Jan-Hendrick to interview him and learn as much as we can from him, as well as potentially working together on an updated equation, since he indicated following December or January

he would be finishing up some work and would be interested to further advance his research and help ours.

It is our intention as a team to begin leaning more on the expertise present within Lockheed Martin as we advance into an equation and mathematics-heavy portion of the LLM Training and Drone Command and Control implementation. We also must do our utmost to find other experts on existing Drone Swarm research to aid us in optimizing the final product, as it would be foolish to not use the many advancements in Drone Search-and-Rescue within even the last 5 or 2 years.

Drone Distance and Measurement

A major consideration of our setup is how our Ground Station interacts with the SOAR and SPARROW team's drones, what limitations exist, and how that shapes, limits, and enables our product design. We must begin by determining the range of movement of drones, in order to find out what mobility is required for the ground station, informing choices of hardware, power supply, and system resilience and reliability in the elements.

Drone Swarm Movement must be tied to transmittable distances to the Ground Station under the agreed upon Minimum Viable Product and can be impacted by a number of factors, such as signal strength over distance. This limitation is providing a scalable limit on Search & Rescue operations when they may occur at a large scale, in mountainous or otherwise transmission-hostile environments, or a combination of the two.

This limitation makes it the case that the placement of the ground Station must be carefully thought out and implemented to maximize drone range in likely search areas. If possible, a mobile ground station would greatly help the ability for this to be less of an issue, and more of a feature in areas where ground troops or aid workers can traverse. This does pose a major issue if drones are being used in a large hazardous area, larger than

the reliable transmission distance, and if the ground station crew cannot traverse close enough.

Because of the need for a mobile Ground Station, the Ground Station would need to run offline, with access to communication with the drones as well as GPS. This can be achieved with a laptop, likely a Windows Operating machine to work seamlessly with the ArduPilot Mission Planner and Ollama systems, and since our team is far more experienced with

Analysis of the terrain and transmutability of signals from the Ground Station to Drones through some sort of sensor or Lidar of some kind would be helpful in determining the limits of Drone Swarm range. We must determine both range from Ground Station, and well as drone-to-drone internal range limitations. In testing phases, using the test drone at first, but hopefully later using the SOAR and SPARROW custom drones in concert, we must ascertain drone-to-ground-Station range, and drone-to-drone range.

Additionally, Drone range will be limited by power consumption during a Search & Rescue mission. Since these drones are going to be designed for continual re-use, they need to return to the Ground Station by the time the Drone dies. In our testing phases, it will be incredibly important to work with the SOAR and SPARROW teams to test the lifespan on the drone batteries in the midst of operations, and measuring maximum safe range of drone operations based upon average drone speed, drone battery life, drone battery drainage rate, and distance calculate when a drone is required to return to the Ground Station, potentially for recharging.

Drone Swarm AI Operating Information

So, establishing how far the drones can reliably function in the testing phase will be an important piece of data to go along with our open-source Drone Swarm AI for any prospective users. In potential dual use cases, like the military, police, FEMA, or any other potential user with established

procedures in place, these specific limitations and end-of-use cases can help the drone swarm AI more readily fit into existing architectures of Search-and-Rescue and be deployed with less hassle on the part of the user, reducing their need to independently test.

This and any other important metadata to include for prospective Drone Swarm AI users would go in a #read-me file on GitHub or wherever else we post the final open-source product, making it easy for users to find and work off of. This other operating information provided alongside the system should include how many parameters the model is trained on, both in the Computer Vision component and the LLM Drone Swarm Movement portion, as well as model data size and training dataset size.

Other operating information can include what methods we used to make, train and fine tune the model, such as reinforcement learning, where the AI Engineers reward the system for good responses, and punish it for undesired responses. We can also include the tech stack involved in the User Interface, making it clear what programs and platforms we've used, allowing greater open source customization from others in the future. One of the most important components of a good open source publication is transparency of process and methods used in the creation and implementation of the product. We can also include information on the drone structure and designs that we used to test.

In addition to a #read-me instructional file, by the end we should produce clear metrics and tests for the Drone Swarm AI, included with the open source project. This can clearly demonstrate the performance of our product, giving an easy understanding for future researchers to determine whether their improvements actually beat out the original model. Visual graphs and models being included to demonstrate these would be incredibly

helpful for user understanding, and for easier comparison testing for future research efforts.

The metrics for testing and future comparison testing should at least include maximum search area in a given time, power consumption, Computer Vision processing time, LLM processing time, Ground Station to Drone signal latency at distance, performance in clear weather vs. Inclement weather (if possible), among others.

Cybersecurity of the Drone Swarm

It may also be prudent to share training datasets we utilized, providing both a way for others to fine-tune the model more easily and expand upon our training in future work, as well as adding transparency to the process of how we made the model. This can also help in the cybersecurity realm, as a clear process of what data made it into our model, when it was introduced, and how can help identify dataset poisoning, model poisoning, sources of prompt injection to the model, or other cyber attacks specific to AI and the training process.

A clear ontology of what data is used and how can also help our own team later on should we work on this from an open-source freelance angle later on, expanding upon our own work. It also makes this more safe and secure for Lockheed Martin to reliably use, as well as the US Government or other organizations that have strict cybersecurity standards as standard procedure.

The defense of the Drone system has multiple identifiable weak points and flaws. For one, the current proposed overall system from the MVP has initial on-board processing on every drone, then sending flagged images to the ground station to further analyze, verify, and then command and control the drones from. Each drone will have installed on-board an NVIDIA Jetson, which increases the cost and unique value of each drone.

Providing Nvidia Jetsons regularly as a requirement for more drones limits the ability to cheaply make mass-produced compatible drones, but the cost is low enough to still enable Search-and-Rescue operations given a small-to-medium budget. If drones are lost, destroyed, or otherwise incapacitated, the supply chain for new drones is bottlenecked by Nvidia stock and supply.

The Ground Station itself is required for the drones to operate properly, so should the Ground Station fail in any significant way, the drones will just hover in place until they run out of battery, or follow a failsafe procedure if we successfully add a functionality for the drones to return to base on loss of connection to the Ground Station.

The drone swarm and ground system communication also have no encryption in the existing plan, meaning in an adversarial use case, information with labelled objects from the on-board Computer Vision processing could be intercepted by an electronic adversary. The drones could also be susceptible to false coordinates sent from an adversarial device using MissionPlanner to subvert the drone swarm.

Most of all, the Drone Swarm uses GPS to navigate, which can be unreliable in stormy and inclement weather or in mountainous areas where signals are blocked. This is the most inherent way for the Drone Swarm to meet failure and potentially lose a drone or fail a mission, and it is unavoidable in the current scope of the project to rectify that vulnerability. The ground station itself should be hopefully running offline, simply using WiFi or other signals to communicate with the drone swarm but not needing to connect to cell towers or other infrastructure like a mobile phone could have to.

Drone Recharging and Replenishment

When operating in the field with the final drones, it would be recommended to have spare drone power banks, or batteries, on hand to recharge spent drones, to reduce the amount of time drones spend on the ground not searching for victims potentially still in danger during the Search and Rescue operation. Having spare drone blades for the roasting chopper portions by the Ground Station alongside the power banks would also be advised for drone repair and replenishment being achieved in an efficient amount of time.

Dependent on the number of resources, it could even be possible that spare drones that finish charging in cycles during “off-time”, when other drone swarms take off to continue the search in cycles. Additionally, the Ground Station, if mobile, must be provided a steady power supply, even if it’s a chargeable battery connected to a laptop, in order to maintain the Drone Swarm’s ability to communicate and scan.

Different drones may have different power requirements, dependent on the makeup of each drone. As we’ve been informed by Joe Rivera, our sponsor, as well as our sister SOAR and SPARROW interdisciplinary teams working on the drones, not every drone might have the same equipment. One drone may have Lidar while another may have infrared camera. If one drone has an NVIDIA Jetson Nano onboard taking care of the YOLO model computation it’s power may drain much faster than that of ones just running a normal camera. This needs to be anticipated and planned for in the hardware setup and charging cycle, as a core component of a computing drone being operational for far less time than other drones could freeze the drone swarm’s ability to efficiently and rapidly cover a Search-and-Rescue area. The potential plan of swapping drones and recharging is a likely solution to this potential logistics and power issue in the drone swarm hardware.

The use of the Drone Swarm in Wild and Urban areas

Since our Drone Swarm would hopefully be able to operate in vastly different areas, containing many different types of objects and identifiable associations for the Computer Vision portion of the model to make, it may require a multitude of various datasets or even versions of the same model to achieve that multi-environment aim. Multiple versions of the model, one for heavy urban, one for suburbs, one for wilderness, one for rural, or just one for populated and one for unpopulated, may be necessary.

This is because of the potential different aims of the Search-and-Rescue, and the different needs of the various applications of Search-and-Rescue. In a wilderness trying to find a lost hiker, it is useful to ignore trees, focus on signs of humans, cars, buildings, smoke, and other hallmarks of civilization. The LLM should be able to identify writing, signs, rocks spelling out "SOS", or other clear cries for help.

Any identification of these signs of the lost hiker would be reported to the ground station with the flagged image, reprocessed by the more powerful Computer Vision portion of our team's model and perhaps shown to a human operator for confirmation, and then process how the drone swarm should move to either confirm a find or search the area.

For our MVP, this is the minimum we need to achieve, this wilderness application of Search-and-Rescue. However if one wanted to search for survivors in a flooded North Carolina suburb, such as following Hurricane Helene, they would want to use heat maps of likely areas based off of buildings, roads, high ground, and civilization, but it would not want to flag every image of an identified building or car to the ground station. This would be far too much information overload, and not be helpful to the overall Search-and-Rescue effort. However, it would want to still identify smoke,

since it may come from an active campfire, people, tents, and other immediate signs of personal presence in the area.

In a heavily urban area, such as during a disaster relief/Search-and-Rescue use case, the LLM and training would have to take into account urban street plans, be able to recognize writing, signs asking for help, or other human visual signals. The presence of buildings and cars would be ignored, and drones would navigate both above streets and above roofs in order to get all angles, searching alleyways and side streets from the top-down, instead of having the camera potentially at an angle in a wilderness use-case. In almost any case it is likely that top-down camera coverage is effective, however any other angle is far less effective in a heavily built-up urban area, such as Brooklyn or Chicago.

However, this demonstrates the need for fundamentally different versions of the Computer Vision portion of the model, so that it identifies the right objects and connects images correctly to the LLM. This also shows the need for the LLM to respond differently in different environments and use cases, following different priorities, patterns, and search algorithms dependent upon the area, previously identified signs of people or people themselves, and other system input.

Potential extended applications of the AI Drone Swarm

This brings up the question of: How easily could this Drone Swarm apply to other scenarios and use cases with a difference in training data and LLM-directed Swarm movement algorithms? There are a berth of organizations and use cases that could utilize an AI-driven Drone Swarm, especially for surveillance, warfare, and coordinated drone efforts outside of Search-and-Rescue.

In the case of surveillance, almost nothing would need to change from the different versions of the models for populated and unpopulated areas,

especially if perhaps guarding and patrolling a secure area, for instance Eglin Air Force Base, which contains a large public wilderness area open to the public, in addition to a great deal of private military ground. To implement it, simply use the nonpopulated area model, search the area regularly for any intruders or suspicious activity, and charge the drones regularly. More specific, single-person identifying surveillance using facial recognition or similar technology would require much more work and entirely separate models, but simply identifying if there are people in an area and what they are doing is a direct application of the Drone Swarm AI.

Additionally, there could be a great deal of dual use in modern drone warfare, especially in the context of the Ukraine conflict where drone warfare is advancing rapidly. In addition to Search-and-Rescue for soldiers who are MIA or civilians, as well as surveillance of military points, a swarm of automated drones that can identify a target to a ground station over a large area at speed is valuable for several reasons.

This potential dual use in a very real and important environment is part of why a clear ontology of data, where we got it, and what tech is involved is very important for secure use of the open source AI drone Swarm in an environment that has active Electronic Warfare in the area. One drawback of the current iteration of the Drone Swarm is that it depends on GPS and MissionPlanner, and famously GPS has been unreliable when blocked by Electronic Warfare in Ukraine and other hostile zones where Search-and-Rescue or other drone operations may be conducted.

Impact in the Search-and-Rescue Community

The Search-and-Rescue community requires the ability to more rapidly, safely, and preferably inexpensively use technology to advance Search-and-Rescue operations. The desire for our product in the MVP, designing relatively cheap drone swarms in an open-source manner that successfully

utilize AI to at least partially automate the drone Search-and-Rescue component, helps in multiple tangible ways.

Currently, according to Jan-Hendrik's research introduction, Search-and-Rescue teams use two human operators for every drone in the air to successfully fly and monitor the video from the drones. Including our AI LLM greatly reduces the need for multiple humans, potentially cutting the manpower needs to run a drone swarm in half. Secondly, the drone swarm working in concert instead of all being flown by individual human operators even more so reduces manpower requirements. Thirdly, building the AI system open-source allows anyone to access the AI, ensuring a rapid acceptance and implementation process for any Search-and-Rescue teams that wish to use it.

Building the Drone Swarm with a budget of \$6300 (the combined budget of our team, and the SOAR and SPARROW team budgets) for 4 drones total, as well as a fully functioning ground station, allows for mass scalability, ease of access, and replaceability. This could allow even normal civilians to afford an unarmed drone swarm with the ability to conduct search and rescue, and puts a mass production of these effectively in the hands of every Federal or State entity in the US, and likely elsewhere. \$100,000 could buy a State Trooper force just under 16 setups matched to ours, allowing for them to find a missing child. Additionally, at larger scales some parts become cheaper, making the supply chain to develop, distribute, and make effective Search-and-Rescue Drone Swarms even cheaper and accessible.

Budget

So far, our group has not used the allotted \$300 budget provided to us by Lockheed Martin and Joe Rivera. We have not had use for it, as any and all training for models that may be compute-intensive in the future could be

done on GPUs present in the Senior Design Lab. The end result required, that of a laptop-based ground station with at most an NVIDIA Jetson Orin running the AI, also emphasizes the requirement for the AI system to run locally and offline on the Ground Station. This restricts model choices to lighter weight models with less parameters, such as Llama models with 7 billion parameters instead of 70 billion parameters.

As for how we might actually use our project budget, it's possible that outfitting our test drone given to use by Dr. Leinecker could be potentially useful in running early tests, but that possibility seems less useful compared to using the actual drones from the SOAR and SPARROW teams. We may get a LiDAR sensor, but that would be it at most. We may use it for training credits should be need to specialize and fine-tune the model using image datasets from drones, beyond what normal school GPUs will be able to do locally on base LLM models and accompanying scripts and prompt engineering.

The use of the budget in the future is far more likely to end up as an extension of the budget for our sister SOAR and SPARROW teams should they run out of their \$3000 budget allotment each and require more without our needing the funds, either for building the drones, software needed for their designs, or more expensive sensors/equipment needed for the SPARROW effort.

Stretch Goals

This project, due to the open-source requirement, lends itself to improvement over time, even without our involvement. However, should we complete our MVP in full early, there are clear stretch goals we may be able to pursue. The Wikipedia for Search-and-Rescue defines multiple areas of the field, including Ground(Lowland), Mountain, Cave, Urban Search and Rescue(USAR), Combat Search and Rescue, and Maritime.

I believe it is not likely for us to be able to add Cave, despite drones being a very good way to safely traverse tight caverns where humans might not be able to go, as that is far beyond the scope of this project. However, should we be able to find Urban, Mountainous, and Ocean datasets that we can train the drone AI on, we could add multiple model versions for different landscapes. The Ocean, for instance, has very different object detection priorities than USAR. To add the variable environments properly, we would also need to calibrate the search pattern priorities, as Urban areas require very different search patterns due to infrastructure.

To combat-proof the Drone Swarm, we could work on three things: Drone Replaceability, GPS, and Encryption. Encrypting the signals between the drones and Ground Station would help protect information the drone relays from being intercepted by adversaries in the Electronic Warfare space. Drone Replaceability could be achieved by having preset designations for drones in a swarm formation, and assigned new drones deployed to replace destroyed or disable drones being assigned to the same designation.

Finally, GPS is the hardest to replace, but could be potentially possible. In Ukraine, due to a lack of GPS because of signal jamming, they have used an AI-assisted camera view called Eagle Eyes, comparing live video from the drone to previously taken photos and video, and determining location from that calculation. If Eagle Eyes was implemented instead of the MissionPlanner software, using the Ground Station as the main computational center, the drones could successfully operate and become far less detectable.

Ollama

Ollama is an open-source AI LLM platform, that functions with Python or Java libraries to run LLMs locally on personal, private devices. It is a Community-driven project founded by two Canadian researchers; Michael

Chiang and Jeffrey Morgan. Ollama provides multiple distinct advantages for our use case and sponsor due to its open source nature. It is not controlled by foreign companies in countries like China or Russia that may be able to access private servers of private companies, protecting US-based sponsors like ours, Lockheed Martin.

The consistent maintenance and updates to Ollama allow for it to be robust against cybersecurity threats, new vulnerabilities, and fresh zero-days against the AI models that can be supported on the platform. The greatest advantage from a security standpoint is that Ollama runs entirely on your private device, and does not send LLM chat information, keystrokes, or any kind of monitoring to external servers, because there are no external servers, in direct contrast to ChatGPT, Gemini, or other private model providers.

Outside of security concerns, there are multiple advantages to our use case. The biggest advantage for the system's security is also the greatest advantage for its usability, mobility and effectiveness: the entire LLM is run locally. The Control & Movement LLM will be able to run offline on the Ground Station, ensuring there is no limit to the environment or a need for Wi-Fi signals due to the LLM itself. Remote Search-and-Rescue efforts in mountainous areas with unreliable signals will be enabled by this system design.

Ollama – Llama 3.2 Vision

In the recent November 6th update to Ollama, support for Llama 3.2 Vision has been added to the framework. Most notably, this includes improved and accurate Image Q&A for image inputs into Llama models in Ollama. As LLaVa, Microsoft's Vision-Assistant model and our LLM of choice, runs on Llama, the newer versions of Llama and LLaVa in Ollama will have

upgraded support for identifying objects in an image natively. This enables our process and workflow of identifying Search-and-Rescue victims directly.

This update also involves Optical Character Recognition, essentially the LLM being able to read written language intuitively. This can allow for accurate recognition of Search-and-Rescue signals, such as “SOS” or “HELP” in large letters, written directions to a last location given by a Search-and-Rescue victim, or other potential clues to a location that can be automatically identified, flagged, and acted upon by the Control LLM.

While not something that is currently identifiable in our workflow, the 3.2 Vision feature that allows for identification of data in models and graphs could be potentially useful if any automatically generated data on the Search-and-Rescue area could indicate more likely locations for a victim to be. This could allow for an additional method of inputting data into the LLM to improve the Search-and-Rescue effort, should relevant graphs and models be made during an operation.

LlaVa and Ollama

Ollama has support for LlaVa models up to LlaVa 1.6, also called LlaVa-NeXT, which includes more resolutions and 4x more pixels, which greatly improves the accuracy and useability of our pixel count algorithms to infer distance from the drone to an identified point using altitude, camera angle, and pixel count. It also has better Image inference and better world reasoning for images in different scenarios, improving the model’s ability to identify Search-and-Rescue targets in a variety of environments. LlaVa’s upgrade to better versions of Llama has enabled this improvement, as well as Open-Source improvements by the LlaVa research team.

First Prototype Installation

To begin the process of creating a prototype Control and Movement LLM, I first got the base models to operationally run locally offline and provide relevant coordinates in an LLM output. I implemented Ollama onto my Windows machine laptop, which is similar to laptop system the SOAR team will use for the ground station. It is an open-source download from ollama.com, that can either be run in its own space or on the Windows terminal.

From there, I used the run <model> command to initiate a download for llama 3.2-7b, which uses 7 billion parameters, making it a lighter model more optimized for a personal computer. I then used a similar command to “run llama” and download libraries for LLaVa afterwards. It then operates a fully functioning LLM within the Windows Terminal. This basic implementation is the foundation we will use for the Control & Movement LLaVa LLM at the core of our project.

Initial tests were promising, as basic prompt engineering to the base LLaVa model could generate new coordinates to move a hypothetical drone to given a starting coordinate set and directions. It unfortunately slightly hallucinated and misinterpreted commands immediately, meaning that we were correct in our assumption that the base model wasn’t specifically trained enough for the use case. I also began simply testing commands and flight paths for one drone, planning to expand to multiple drones and calculating optimal areas of camera coverage and flight plans later, first wanting to identify the ability to accurately move a drone on a coordinate plane.

For example, in response to “You are a drone hovering at 70 ft. Above a flat plain, at the coordinates 41N 31W. Move the drone 50 ft. North and provide new coordinates.” In response it gave a wordy explanation, proceeded to subtract the elevation by 50, equaling 22, which is incorrect

and unnecessary. It provided the Latitude 41.03516 and Longitude 30.98352, which while it moved the drone west in addition to North unnecessarily. Following this initial test, It successfully gave similar coordinates in the correct direction when I commanded it to "Move west 50 feet. Provide only new coordinates" in an LLM prompt, however it still provided text in addition, which would mess up the inputs to MissionPlanner directly from the LLM output.

Model Improvements

There are multiple methods required to fix the problems with the initial base model. Firstly, the response needs to just provide Coordinates, in a format readable to the ArduPilot MissionPlanner application that the Ground Station will have installed to functionally control and communicate directions to the drone swarm. This will be achieved by implementing tool calling onto the Ollama API and our model. Ollama has pre-existing support for tool calling, including support specifically for tool calling to the llama 3.2 model, that LLaVa will run on, as described on the Ollama models page.

Secondly, when we run the LLaVa model in the terminal, we can add python (or Java) scripts which will provide context, AI Guidelines and Guardrails for the LLM Output in the form of prompt engineering, as well as specific metadata fields input by a human operator. These fields would include Search-and-Rescue operation coordinate limits bound to the active search area, and as LLaVa 1.6 can read graphs and map overlays, it would be possible for the LLM to read input graphs of a new search area and recalculate a search path for it. This would require the script to be updated from inputs in the LLM prompt field given the correct python script and file setup. This would likely be achieved through a customized version of an Ollama UI plugin that allows script updating and LLM reloading with the updated script.

We also need to learn proper prompt engineering and how to effectively implement AI Guidelines and Guardrails into the model. We have researched how to properly frame prompts to the end UI, and providing instruction and simple training for human operators on how to best communicate with the LLM in the field would be a useful addition to the project. This would likely be in the form of information in the Github README file, showing suggested prompt formats.

Then, using the MITRE ATLAS framework for AI & Cybersecurity, I found guidance for guidelines and guardrail mitigations against model abuse, inaccuracy, and hallucination. The LLM Guidelines, safety controls placed onto the LLM through a Python script, append onto user inputs and use affirmative language to enforce answer standards and outputs. This step occurs between the user input and the interpretation of the input, changing the input. The LLM Guardrails, safety controls also enforced through the python script, change the output during and after it has been processed by the LLM, screening for inappropriate or out-of-bounds responses.

Utilizing both methods of LLM control in our python scripts, in addition to automated tool calling, should effectively provide a secure and reliable LLM output that is readable to MissionPlanner. This output can also be modified for other flight programs with a modified Python script, making this an effective open-source project that can be adaptable to multiple drone systems.

Native Drone Computation

Also, as of November 18th in the Ollama 0.4.2 update, Ollama now provides support for Nvidia Jetsons and can be installed onto Jetsons with the command “curl -fsSL <https://ollama.com/install.sh> | sh”. As the current plan from the SOAR and SPARROW teams is for each of the drones to have an on-board Jetson Xavier or Nano for computing, this now allows for the

possibility of no Nvidia Jetson Orin- supported Ground Station in a future design, or more likely the Ground Station simply having the ability for human operators to recall the drones or send prompts to the LLM running on the drones.

A likely implementation of this would look like a central command drone with a Jetson installed running the LLM and sending commands to others, making the others lower cost with less Nvidia Jetsons required to launch the swarm. However, especially in battlefield Search-and-Rescue use cases or in heavy weather conditions where drones may be endangered, it is useful to have redundant Jetsons on multiple drones that can transfer Command of the drone swarm seamlessly upon a drone's destruction or malfunction.

Native Drone Computation, without the need for Ground station computing, supports potentially faster compute and communication times, as the drones don't need to communicate with the ground station and send larger image or video files between it and the swarm. The native computation also allows for more secure communications between drones and human operators. Now direct coordinates are only communicated on and between the drones, not over a presumably larger distance that could expose future coordinates to enemy Red Teams.

Training and Reinforcement Learning

The Ollama-based LLaVa model will run on Llama 3.2 at time of writing, and unless there are major developments in the next 2-3 months for a Llama 3.3, this is the final tech stack for the LLM for our project release, presentation, and publishing online

With the LLM tech stack finalized, we now need to match model training to best fit our components of choice. The stages of training a LLaVa model include data preparation, pre-training, training, and testing. For data

preparation, we gather a custom dataset of top-down drone camera images, including a sufficient number of images in groups of variable environments. We need to have several forest or plains images, a dataset of urban/populated images, a dataset of ocean or bodies of water images.

Then in pre-training, the python scripts that contain model-ingrained prompt engineering, guardrails, and guidelines must be custom-fit to each environment we have separate data for. In the final product, if we achieve multiple environments, the human operator should be able to tell the model UI which environment it is operating in, so it uses a version of the model trained for that environment.

The data preparation of these datasets should involve labelling them for what the correct image associations are, to train the model to correctly recognize objects and people. We can use services like UBIAI to automatically label images for the objects contained, manually check their accuracy, and add them to the dataset the model will use.

There are several optional and variable methods to achieve the training and testing, and we believe one of our best options for the use case is reinforcement learning. Reinforcement learning involves setting up the model in a test environment, giving it test inputs, and giving it a graded reward based upon the success of its output. The system optimizes its responses to get the maximum reward the test environment will give, training the model to fit the test criteria. Each time the system gets a reward, it will update its policy or value function, which maps a state, like s to an action a .

Reinforcement learning can happen in or outside of a model environment, and due to our timeline, we will likely use our model testing interface developed by Brody to track progress, but otherwise simply run the model on Windows terminals, matching the use case conditions.

Reinforcement learning can also have pitfalls, however. If the data is not variable enough, the model will match to fit the limited data given and make false associations. It might ascertain that a birch tree means there are humans if all the photos with humans happen to have birch trees whereas others don't, for example.

To mitigate the issues with reinforcement learning, I believe it is best to use a new form of automated reinforcement learning: Reinforcement Learning with AI Feedback (RLAIF). Normally, Reinforcement Learning with Human Feedback (RLHF) is used to have human operators determine if it was a good response or not. RLAF allows a secondary AI model to analyze the output against an expected output. This is far more accurate once set up due to the automated output comparison, and because RLHF requires a lot of human feedback to be accurate. RLHF operates using a preference model, asking the human's preferences on the output after receiving it. The best version of this is ranked preference, being shown two outputs from the LLM on the same prompt and picking the better one.

RLAIF however, operates by having the secondary model generating it's own set of Ranked Preferences it will judge the original model on, which can be fine-tuned by our team to ensure it matches the priorities of identifying humans, identifying traces of civilization, and tracks. This adds associate preference scores as data labels to the responses, automating the Reinforcement Learning process.

This can be monitored by our Senior Design team by saving the data preference labels as metadata each training epoch, allowing us to determine the effectiveness of the AI judgement model. This informs us when to change or undo any training done with the AI Feedback, should the preference labels begin favoring unrealistic or incorrect patterns.

To use this Reinforcement Learning with AI Feedback for our model, we need a model that will provide that feedback. I have identified an open-source model for this purpose: starling-lm. Starling-lm is an open-source LLM on the Ollama framework that has been itself trained with Reinforcement Learning and AI Feedback. It is designed to take an LLM Chatbot's outputs and provide feedback to improve the "helpfulness" metric. The model can be run in Ollama on the same Windows Terminal as our Control LLM by using the "ollama run starling-lm" command. We can iterate multiple terminals running the models simultaneously, input the Control LLM's outputs, set up the AI Feedback model to provide a weighted reward based on its generated Ranked Preferences, and allow the automated feedback to improve the model's helpfulness, which includes accuracy and relevance of responses.

As we run multiple epochs of training the Control LLM, the automatic flow of model outputs to the starling-lm model and reward responses will shape the original Control model, successfully achieving the training step of developing the Control and Movement LLM. The training phase will overlap with the testing phase, as flaws identified in the testing phase will result in a return to more training. Testing the model already includes one of the steps of reinforcement training, automating a reward for good and bad responses. But the larger testing phase will involve manual post-epoch terminal tests, simulation tests, and eventually field tests.

Manual terminal tests after each epoch will allow us to find traditional AI metrics and results, such as model performance, and our own metrics, such as % accuracy of how many humans were correctly identified in dataset images. These can be run natively on our Windows systems and terminals with minimal effort on a regular basis throughout the training process and immediately after training, allowing us to find issues and problems without setting up a field test or simulation.

Small-scale field tests are now available to us for a single drone with a camera due to the drone donated to us by Dr. Leinecker, and can be useful in seeing how early versions of the model perform. As our sister teams SOAR and SPARROW finish their drone and ground station development, we will be able to upgrade to swarm field tests, as well as specialized tests to account for drones with different equipment and sensors. As we near the end of the course, these integrated field tests will involve fully loading the model into a fully functioning Ground Station to demonstrate the project is successful.

Simulation tests should become available to us mid-Spring semester, as our 3rd sister team is working on a simulation in Unreal Engine for a drone swarm. Once it is live and usable for our team, we will be able to quickly and efficiently load our LLM into the environment, and use it to simulate a number of versions of our LLM's prompt engineering, training, and model versions in an easy and cost-effective manner. The simulation tests, depending on timeline and the effectiveness of other groups, may allow us to test the effectiveness of different search algorithms before we have a physical drone swarm.

Transitioning from Single Drone to Drone Swarm Model

The current prototype model setup and training is intended to achieve a successful single drone taking commands and changing coordinates due to successful identification of stimuli in input images. The expansion to multiple drones taking commands and moving to synchronized positions still needs to be added. The current prototype also fails to actually consider a search algorithm or a proper flight path, but instead is focused on successfully outputting an accurate set of coordinates given an instruction.

The LLM needs to upgrade from a follower being instructed where to go, to encapsulating the entire workflow. The LLM needs to infer coordinates from image-pixel equations, access potential search algorithms, determine

the best algorithm for the situation, run the search algorithm, translate that output to sets of coordinates, assign them to individual drones, and successfully send them to the drone swarm. It currently possesses none of that new functionality.

Multiple drones also require 2 additional steps to determining drone coordinates: multi-drone area coverage algorithms and swarm movement algorithms. The LLM needs to call functions that run these automated algorithms and determine what drone paths will be the most efficient. We also want to add the ability to switch to heat maps as a stretch goal on top of this.

To implement the multi-drone area coverage algorithm, we need to adapt the pixel-distance algorithms we have developed and instead calculate what area the drone camera is currently covering, determining the coordinates the camera is viewing between based off of the drone's current coordinates and altitude. We then need to keep a running record of what areas the drone cameras have covered so far, so that the LLM doesn't retread the same ground unnecessarily.

The most accurate version of implementing this would be implementing a model of the landscape in a plugin connected to the LLM, at least in 2d, and "highlighting" or drawing over the coordinates where the drones have covered thus far. The LLM can then use this map that updates in real-time as a new temporary bounds for the Search-and-Rescue operation area, ensuring that the Control LLM will always be calculating the flight paths with the most current information.

Search Algorithm Implementation

To implement the swarm movement algorithms, we need to have several accessible python functions that contain the different deterministic and non-deterministic algorithms, and at a standard interval during the

search, perhaps every 1 or 5 minutes, run each of those algorithms to determine key metrics. These include metrics such as: total time to completion, % area coverage within 1 hour, and % area coverage within 3 hours. The different % area coverage metrics will help show how fast the algorithms can cover a majority of the area, and prioritizing general area coverage earlier in a Search-and-Rescue effort is a highly recommended tactic to find endangered people faster.

The LLM should then apply weights to these metrics, prioritizing early % area coverage and total time to completion, and select the search algorithm or technique that performs the best to continue the search with. Included in this process should be event-based algorithms for localized searches, trail following, or similar custom actions. For instance, upon identifying a trace of civilization, like a tent, a spiral localized search pattern using all drones would be initialized. Implementing these event triggers will occur in the LLM Python scripts, ensuring that when it identifies these parts in an image it will be trained to respond with the correct localized search algorithm.

Once the correct search algorithm is selected after each regular interval, the LLM needs to constantly evaluate the next drone movement coordinates for each individual drone along the search pattern. A wrinkle in the search algorithm plan is that not every drone might be the same. While we have been assured every drone will have a camera, only one drone might have LiDAR, while one other might have thermal imaging. This means, to take advantage of these specialized sensors, drones with very different equipment may have to cover the same area multiple times, in case the thermal imaging picks up something the camera does not, for instance.

Accounting for this is important, and our solution is to develop “drone roles” with different search paths and priorities. In essence, splitting the

swarm into multiple swarms and individuals. If we have 4 drones, all with cameras, one with LiDAR and one with a night vision camera, there would be one larger search algorithm for the drones with only a camera, and individual search paths for the specialized equipment later in the process.

In this scenario, since all drones have cameras, all drones would scan in one distributed search pattern to cover as much ground as possible. As the covered area of the search increases, the need for specialized drones to retread the same area increases. The drone with night vision would be a special case as well, as it only counts as a “specialized drone” at nighttime, and during the day will always be part of the larger swarm search. We will need to work with SOAR and SPARROW to determine what sensors are confirmed beyond cameras and LiDAR, what the drone specializations are, and then independently research how to best deploy those specialized drones in the swarm.

Expected End Results

For this project, we have many aspirations for how far we can apply the model, in multiple environments, in a secure manner, and with custom specialized drones. But what we are most focused on is allowing 3-4 drones to search a wilderness area guided by an LLM, have that LLM effectively communicate with the Drone Swarm through MissionPlanner, and have that successfully LLM successfully identify a human without false positives.

Beyond that, we are striving to teach the LLM to compute and use the most efficient search pattern possible. We plan to achieve an easily replicable system that is open-source and readily available for the Search-and-Rescue community to use. The intent of this project from our Lockheed Martin sponsor was to find an effective low-cost solution, and should we train our LLM well enough, it should be a safe and effective option to use in real-world Search-and-Rescue and Disaster relief scenarios. If this project in

any way helps people or saves lives, it will have been the honor of a lifetime to have done so successfully.

Ricardo's Section

Updated General Requirements of the AI agent

Once we met with our sponsor, certain requirements were changed, and new objectives were defined. While the original stated objectives remained largely unmodified, some new additions were added to better align with the project's goals. The focus of these updates was to streamline our goals and ensure that our Search and Rescue AI agent would be capable of fulfilling the needs of real-world applications.

The first key requirement was that the LLM must be able to consistently conduct operations without encountering critical errors that could cause it to crash or exist unexpectedly. Stability is important, since the AI agent would be working in real-time environments where failure is not an option. Should the stability fail, the LLM's functionality could be reduced to a point where mission failure would occur and potentially lead to the target not being found.

The second key requirement was that the LLM retain the knowledge and skills it acquired during previous training sessions, allowing it to perform the search and rescue operations confidently. This meant that the system needed a way of remembering how to effectively conduct these operations, learning from past operations and improving itself over time. This memory would ensure that the LLM didn't have to relearn everything from scratch, saving time.

The third key requirement was that the LLM needed to be able to identify specific objects at great distances. The majority of the drone's